SWENG 894 Capstone

MicroMatch

Software End-User Manual

Chastidy Joanem

M.S. Software Engineering | April 2026

[MicroMatch Repository Link]

Table of Contents

Section 1: Product Overview and Deployment

1.1 Product Overview

MicroMatch is a web-based micro-internship marketplace that connects university students with organizations offering short-term, skill-based projects. The platform provides a complete end-to-end workflow from project discovery and application through deliverable submission, project completion, and post-project feedback.

The system supports three user roles:

- **Students** — discover projects, apply, submit deliverables, exchange messages with organizations, and earn achievement badges based on completed work.
- **Organizations** — post project listings, review and accept or reject student applications, review submitted deliverables, mark projects complete, and message accepted students.
- **Administrators** — monitor platform activity through system logs and moderate content.

Key platform capabilities include personalized smart recommendations powered by a weighted matching algorithm, a real-time messaging hub scoped to each project, an analytics dashboard showing productivity metrics per role, an auto-computed badge system for student achievements, and a notification bell that alerts users to all key events.

1.2 System Architecture Summary

Layer	Technology	Role
Frontend	React 19 (Create React App) + Tailwind CSS	SPA served on port 3000. All user interactions.
Backend	FastAPI (Python 3.11)	REST API served on port 8000. Business logic and RBAC.
Database	PostgreSQL 15	Persistent storage. Provisioned via Docker Compose.
Auth	JWT (python-jose)	Stateless token-based authentication.
Containerization	Docker + Docker Compose	Spins up PostgreSQL for local development.

1.3 Required Tools, Frameworks, and Services

The following tools must be installed on your machine before deploying MicroMatch. All are free and open-source.

Tool	Version	Purpose	Download URL
Git	Any recent	Clone the repository	git-scm.com/downloads
Python	3.11+	Backend runtime	python.org/downloads
Node.js	18+	Frontend build tool (npm)	nodejs.org/en/download

Tool	Version	Purpose	Download URL
Docker Desktop	Latest	Runs PostgreSQL container	docs.docker.com/get-docker
Docker Compose	v2+	Orchestrates database (bundled with Docker Desktop)	Included with Docker Desktop

Note: Docker Desktop includes both Docker and Docker Compose on Windows and macOS. On Linux, install docker-compose-plugin separately via your package manager.

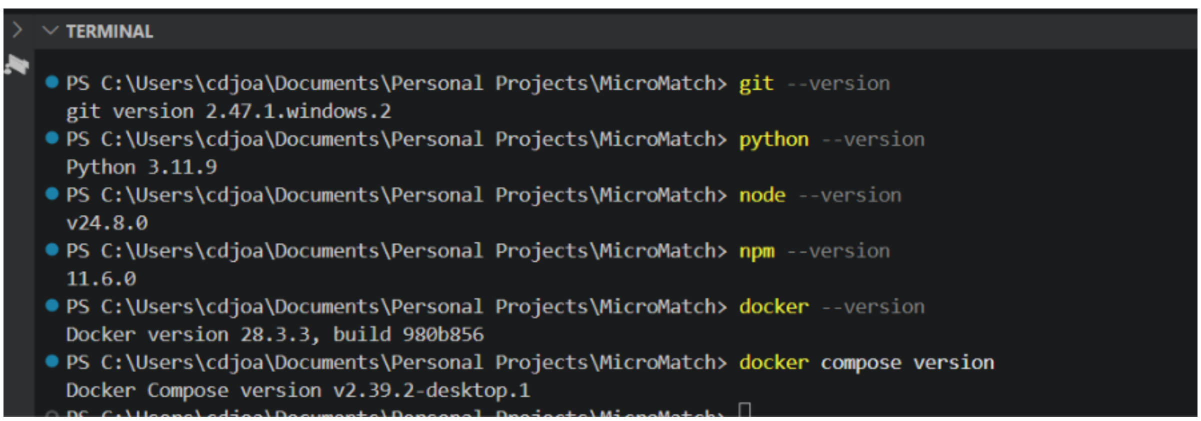
1.4 Deployment Instructions

Follow these steps in order to deploy MicroMatch on your local machine.

Step 1 — Install Prerequisites

Install all tools listed in Section 1.3. Verify each installation from your terminal:

```
git --version
python --version
node --version  &&  npm --version
docker --version  &&  docker compose version
```

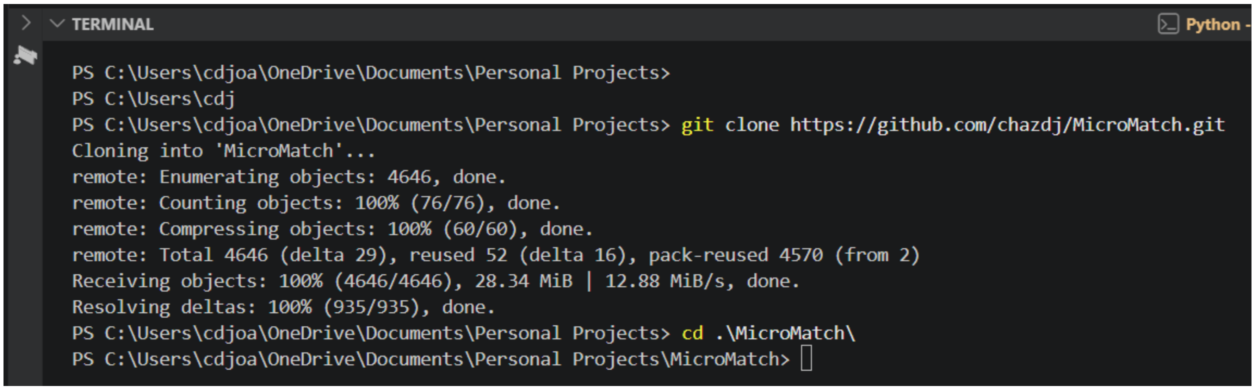


Terminal output confirming all prerequisites are installed correctly.

Step 2 — Clone the Repository

1. Open a terminal (Command Prompt, PowerShell, or Terminal on macOS/Linux).
2. Navigate to the directory where you want to store the project.
3. Run the clone command:

```
git clone https://github.com/chazdj/MicroMatch.git
cd MicroMatch
```



Step 3 — Start the PostgreSQL Database

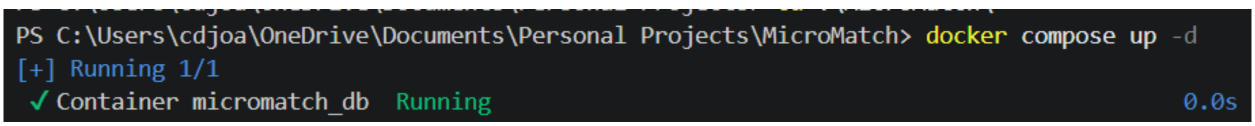
MicroMatch uses Docker Compose to provision a PostgreSQL database.

1. Make sure Docker Desktop is running (you should see the Docker icon in your system tray).
2. From the project root directory, run:

```
docker compose up -d
```

The **-d** flag runs the container detached. Verify it is running:

```
docker compose ps
```



Docker Desktop showing the PostgreSQL container with status Running.

> **Note:** The database schema (all tables) is created automatically the first time the backend starts. No manual SQL setup is required for the initial tables.

Step 4 — Configure the Backend Environment

1. Navigate to the backend directory:

```
cd backend
```

2. Create a Python virtual environment:

```
python -m venv venv
```

3. Activate the virtual environment:

```
# Windows: venv\Scripts\activate
# macOS / Linux: source venv/bin/activate
```

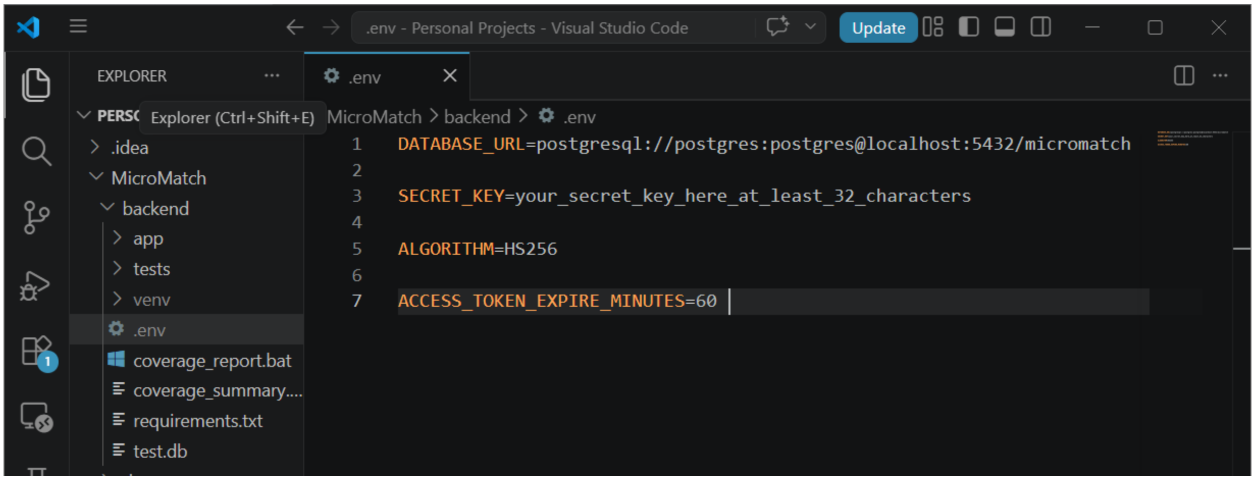
4. Install dependencies:

```
pip install -r requirements.txt
```

5. Create a **.env** file inside the **backend/** folder with these contents:

```
DATABASE_URL=postgresql://micromatch_user:micromatch_password@localhost:5432/mi
cromatch
SECRET_KEY=your_secret_key_here_at_least_32_characters
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
```

> **Note:** Replace the SECRET_KEY value with any random 32+ character string. Generate one with: python -c "import secrets; print(secrets.token_hex(32))"



Step 5 — Apply the Database Schema Migration

The table schema is auto-created on first startup, but two profile enhancement columns must be applied manually:

```
psql -U micromatch_user -d micromatch -c "ALTER TABLE student_profiles
  ADD COLUMN IF NOT EXISTS portfolio_links TEXT,
  ADD COLUMN IF NOT EXISTS badges TEXT;"
```

> **Note:** If this command fails because the table does not exist yet, complete Step 6 first (start the backend), then return and run this command.

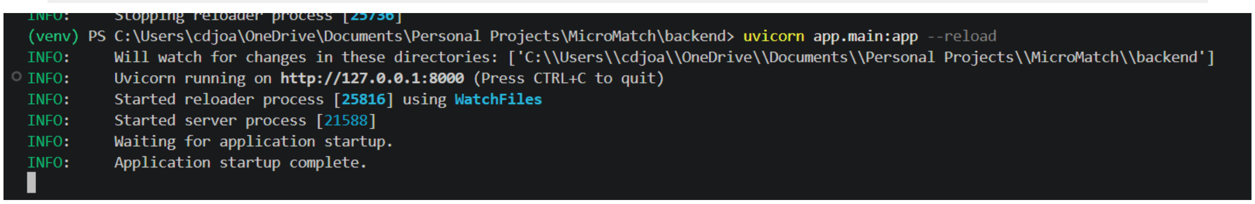
Step 6 — Start the Backend API

With the virtual environment active and .env file in place:

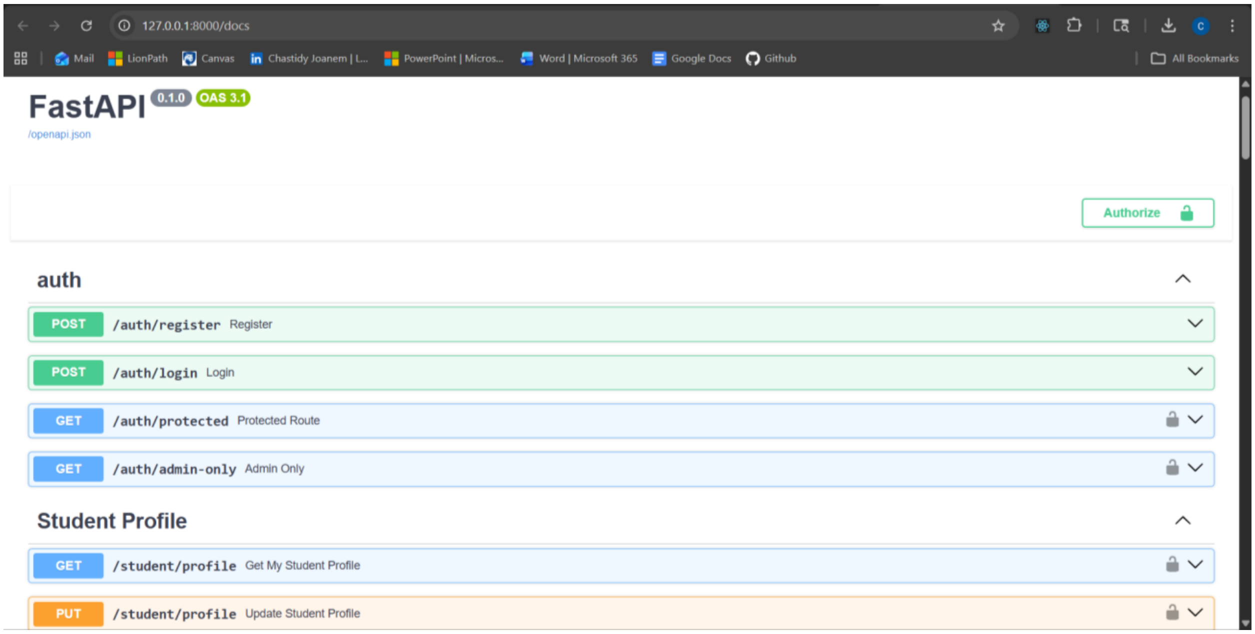
```
uvicorn app.main:app --reload
```

You should see the following output:

```
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Application startup complete.
```



The interactive API documentation is available at: **http://127.0.0.1:8000/docs**



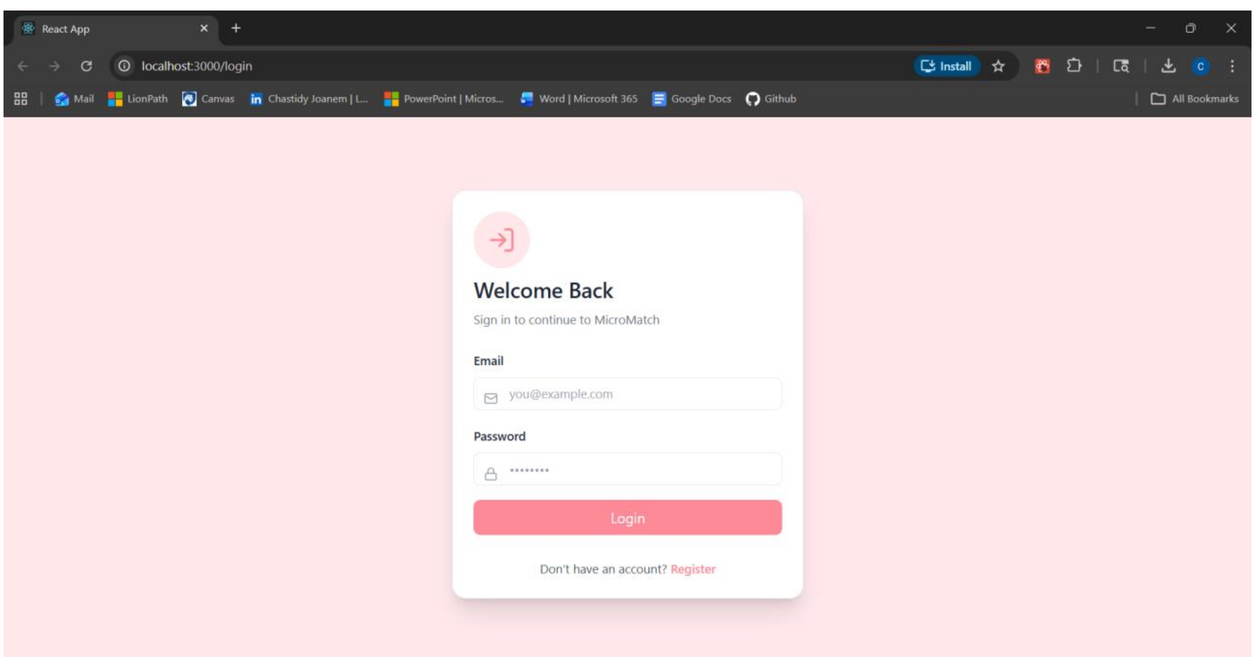
FastAPI automatically generated Swagger UI at localhost:8000/docs.

Step 7 — Install and Start the Frontend

Open a **new terminal window** (keep the backend running in the first terminal):

```
cd frontend
npm install
npm start
```

The React application opens automatically in your browser at **http://localhost:3000**.



MicroMatch landing page at http://localhost:3000. The application is ready to use.

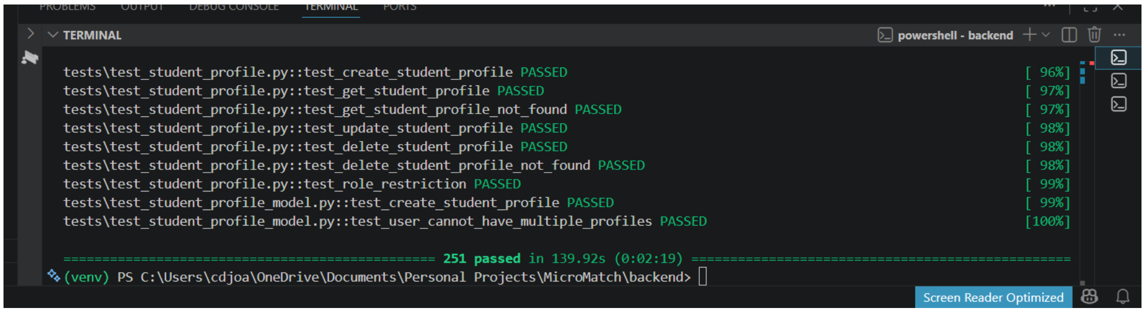
> **Note:** Keep both terminal windows open simultaneously: one running the backend (port 8000) and one running the frontend (port 3000).

Step 8 — Run Backend Tests (Optional)

To verify the backend is correctly configured, run the automated test suite:

```
cd backend
pytest -v
```

All 251 tests should pass. Test execution takes approximately 2-3 minutes.



1.5 Quick-Start Summary

Command	Description
docker compose up -d	Start PostgreSQL database
cd backend && source venv/bin/activate	Activate Python environment (macOS/Linux)
cd backend && venv\Scripts\activate	Activate Python environment (Windows)
uvicorn app.main:app --reload	Start API server (port 8000)
cd frontend && npm start	Start React dev server (port 3000)
http://localhost:3000	Access the application in your browser
http://localhost:8000/docs	Access the interactive API documentation

Section 2: Application Features and Instructions

2.1 Feature Summary by Category

MicroMatch organizes its functionality into six categories. All features are accessible from the navigation bar after logging in.

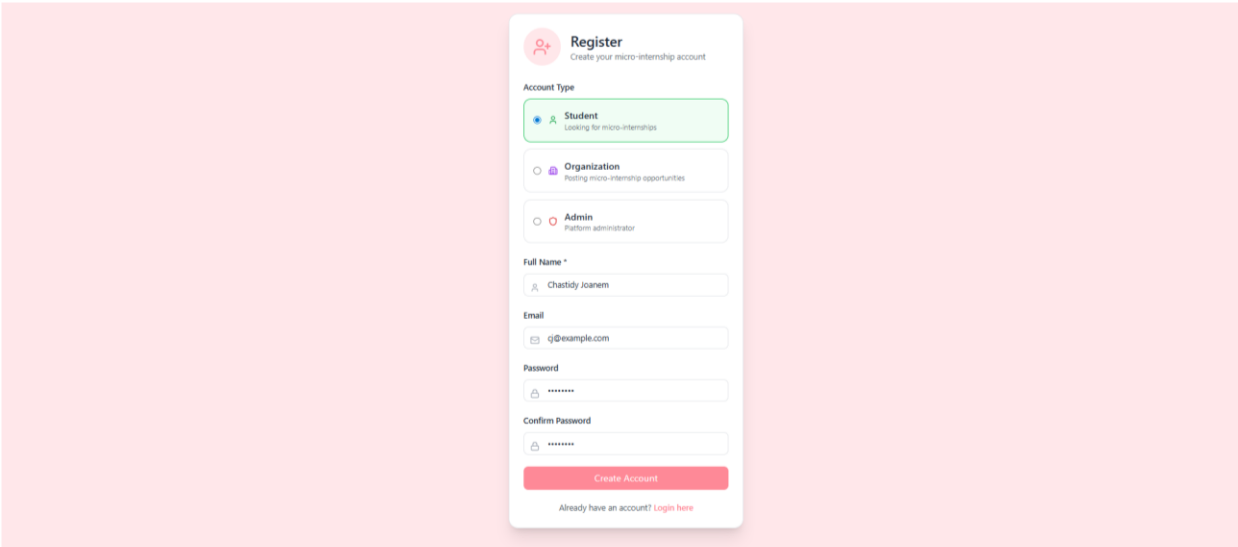
Category	Features	Available To
Account Management	Registration, Login, Logout	All users
Profile Management	Create/edit student profile with skills, bio, portfolio links; organization profile; auto-awarded achievement badges	Student, Organization
Project Management	Post projects, browse/search listings, view project details, smart recommendations	Student (browse), Org (post)
Application Workflow	Apply, track status, accept/reject applicants, submit deliverables, review deliverables, mark complete, feedback	Student, Organization
Communication	Project-scoped messaging hub, notification bell for all key events	Student, Organization
Analytics	Analytics dashboard with stat cards and bar chart; role-separated views	Student, Organization
Administration	System request logs, content moderation	Admin only

2.2 Account Management

2.2.1 Register a New Account

New users must create an account before accessing any platform features. MicroMatch supports three roles: Student, Organization, and Admin.

1. Navigate to **http://localhost:3000**.
2. Click the **Register** link on the login page.
3. Fill in your **Full Name**, **Email Address**, and **Password**.
4. Select your **Role** from the dropdown: Student, Organization, or Admin.
5. Click **Register**.
6. You will be redirected to the home dashboard automatically after successful registration.

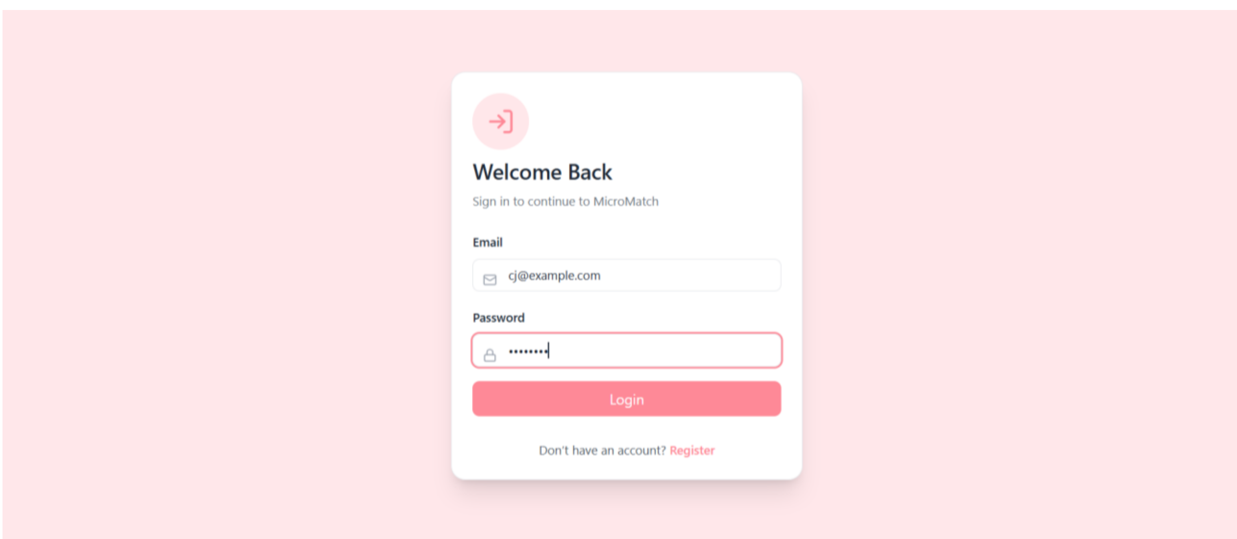


Registration form. Select the correct role before submitting — it cannot be changed after registration.

Note: Your role cannot be changed after registration. If you select the wrong role, create a new account with a different email address.

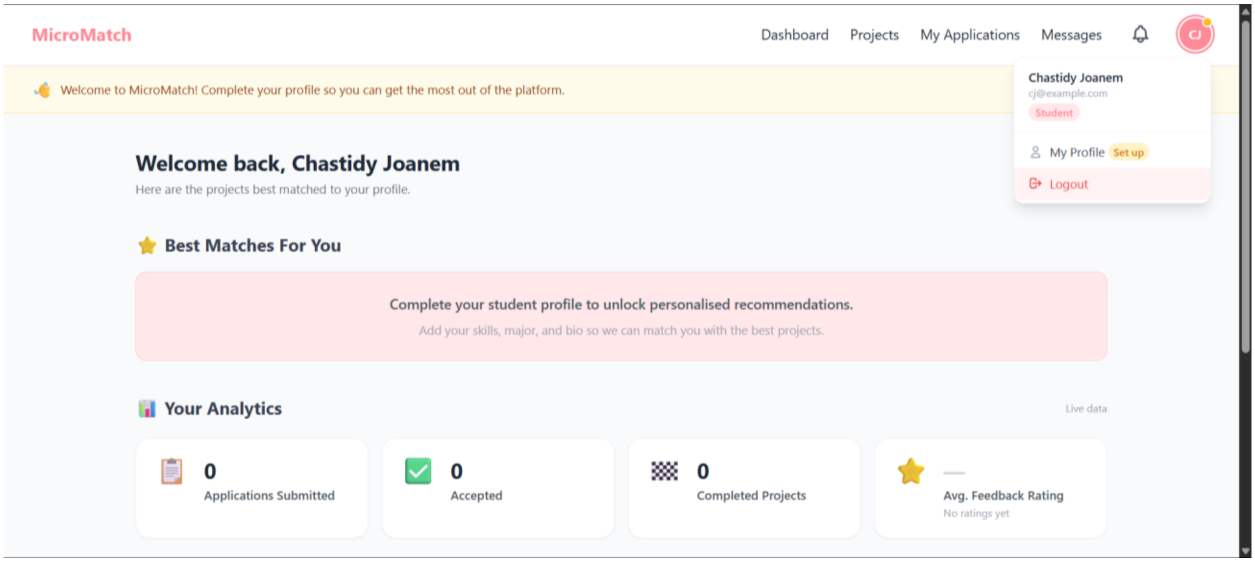
2.2.2 Login

1. Navigate to **http://localhost:3000**.
2. Enter your registered **email address** and **password**.
3. Click **Login**.
4. You will be taken to your role-specific home dashboard.



2.2.3 Logout

1. Click your **profile avatar** in the top-right corner of the navigation bar.
2. Click **Logout** from the dropdown menu.
3. You will be redirected to the login page.

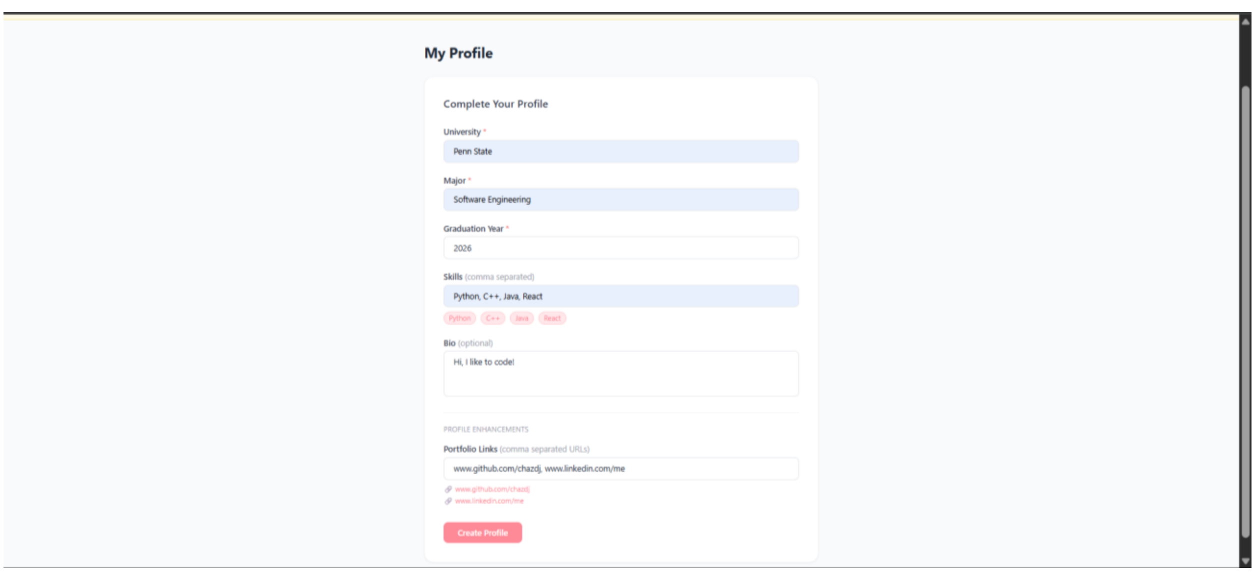


2.3 Profile Management

2.3.1 Create a Student Profile

After registering as a student, a banner prompts you to complete your profile. A completed profile is required to apply to projects and appear in smart recommendations.

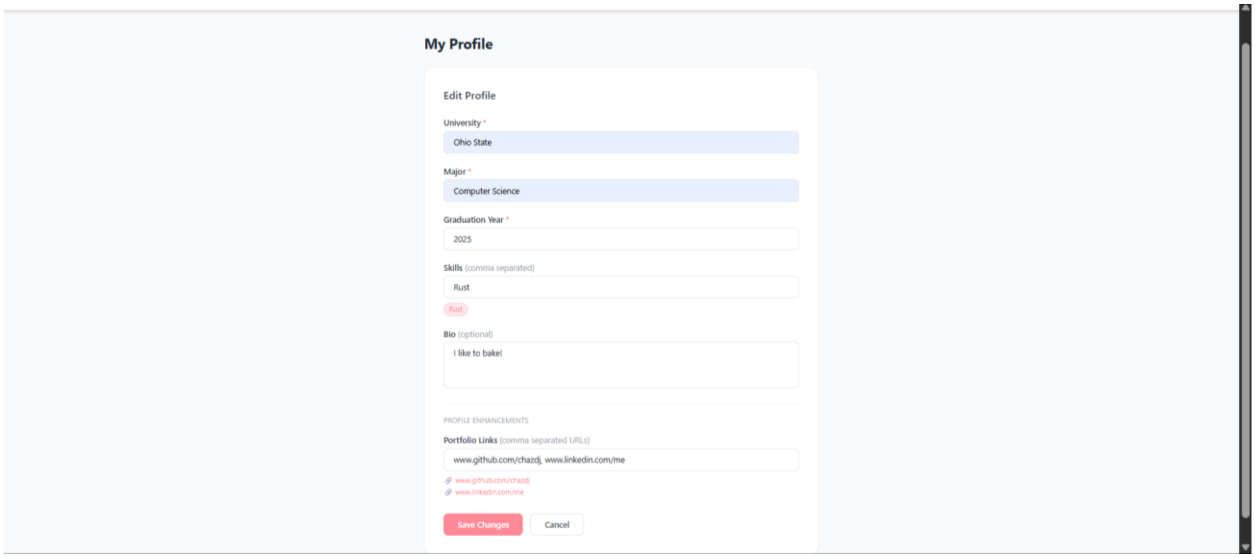
1. Click **Complete Your Profile** from the home dashboard banner, or navigate to **My Profile** in the navigation.
2. Fill in your **University**, **Major**, and **Graduation Year** (required fields).
3. Add **Skills** as a comma-separated list (e.g., Python, React, SQL). Skills appear as colored chips in view mode.
4. Write a short **Bio** to introduce yourself to organizations (optional).
5. In the **Profile Enhancements** section, add comma-separated **Portfolio Links** (e.g., https://github.com/yourusername).
6. Click **Create Profile**.



Student profile creation form. Skills entered as comma-separated values preview as chips in real time.

2.3.2 Edit a Student Profile

1. Navigate to **My Profile** from the navigation bar.
2. Click **Edit Profile** in the top-right of the profile card.
3. Update any fields. Skill chips preview live as you type.
4. Click **Save Changes**.



Student profile in view mode. Skills display as chips, average rating shown as stars, and auto-awarded badges appear in the Achievements section.

2.3.3 Achievement Badges (Auto-Awarded by the System)

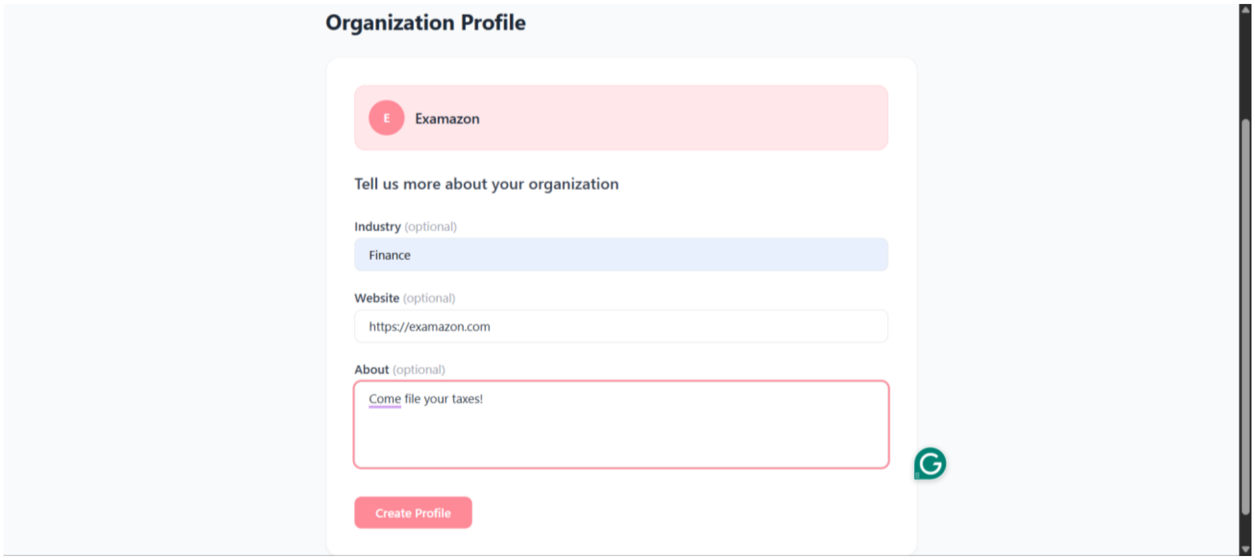
Badges are awarded automatically based on verified activity. Students cannot manually add, edit, or remove badges.

Badge	Award Criteria
First Project	Complete 1 project (accepted application + project marked complete)
3 Projects	Complete 3 projects
10 Projects	Complete 10 projects
Top Rated	Average feedback rating >= 4.5 stars across all completed projects
Rising Star	3 or more completions AND average feedback rating >= 4.0 stars

> **Note:** Badges appear automatically in the Achievements section of the student profile after the triggering event. No action is required from the student.

2.3.4 Create an Organization Profile

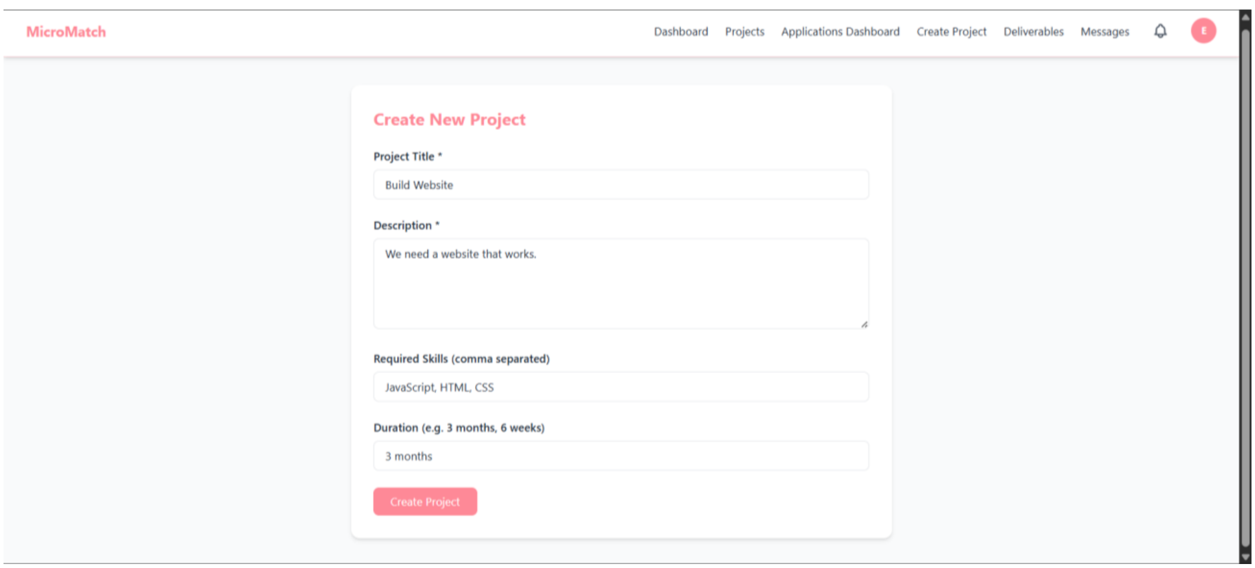
1. After registering as an Organization, navigate to **Organization Profile** from the navigation.
2. Enter your **Organization Name**, **Industry**, **Website**, and **Description**.
3. Click **Save Profile**.



2.4 Project Management

2.4.1 Post a New Project (Organization)

1. Click **Create Project** in the navigation bar.
2. Fill in the **Project Title**, **Description**, **Required Skills** (comma-separated), and **Duration**.
3. Click **Post Project**.
4. The project is immediately visible to students on the Projects page.

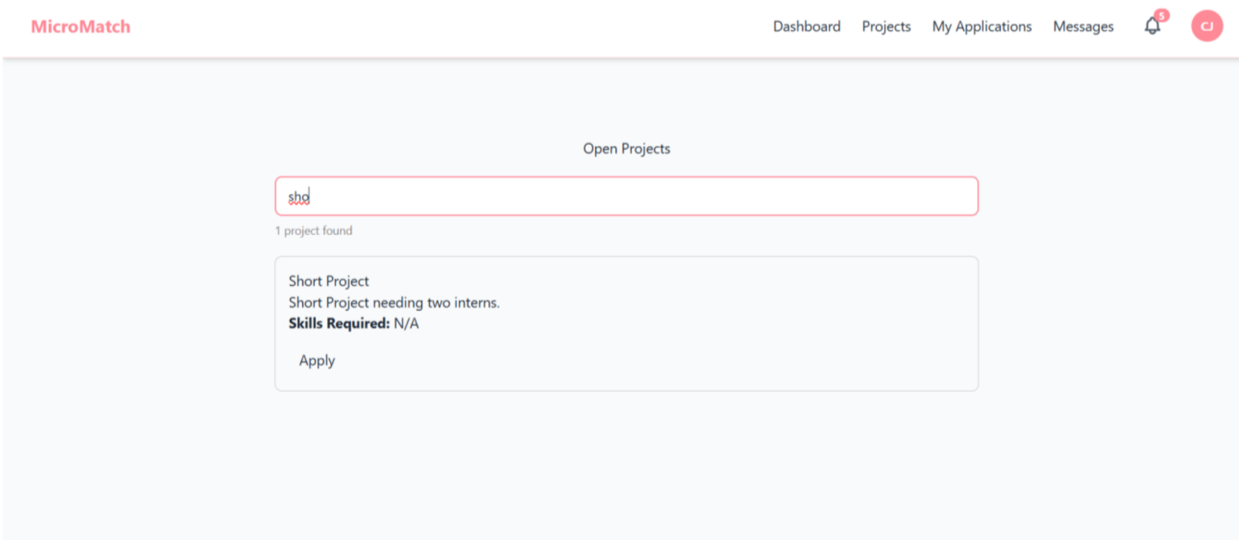


Organization project creation form. Required skills are used by the smart matching algorithm to rank and recommend projects to students.

2.4.2 Browse and Search Projects (Student)

1. Click **Projects** in the navigation bar.
2. All open projects are listed with their title, description, required skills, and duration.

3. Use the **search bar** to filter projects by keyword (matches project title, description, and required skills).
4. Click any project card to view the full description.
5. Click **Apply** to submit an application from the project detail view.



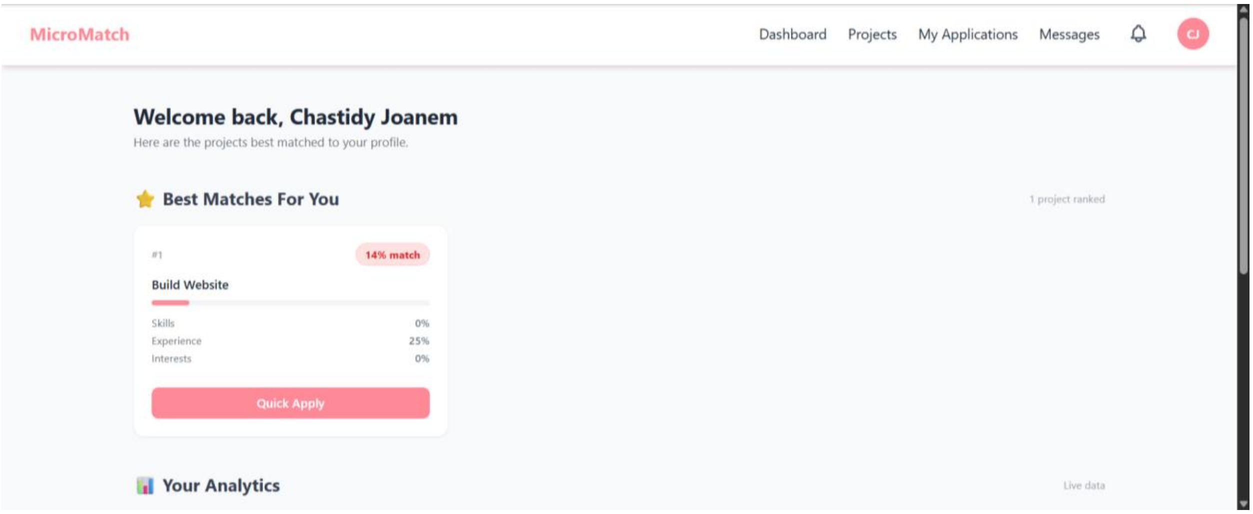
Projects browse page. Each card shows required skills as tags to help students identify matching opportunities.

2.4.3 View Smart Recommendations (Student)

The home dashboard displays projects ranked specifically for you by a five-factor weighted matching algorithm. Each project receives a score (0-100) based on:

- **Skill match (40%)** — Coverage of the project's required skills by your listed skills.
- **Experience Match(20%)** — Proximity of your graduation year to the project's timeline.
- **Interest Match (15%)** — Keyword overlap between your major/bio and the project description.
- **Activity score (10%)** — Penalizes projects you have already applied to.
- **Success Score (15%)** — Your historical track record: completion ratio, deliverable acceptance rate, and average feedback rating. New students receive a neutral baseline score of 50/100.

1. Log in as a student with a completed profile (university, major, graduation year, and skills).
2. The **Best Matches For You** section on the home dashboard displays ranked project cards.
3. Each card shows a match score percentage and a breakdown of the five contributing factors.
4. Click **Quick Apply** on any card to apply to that project immediately.

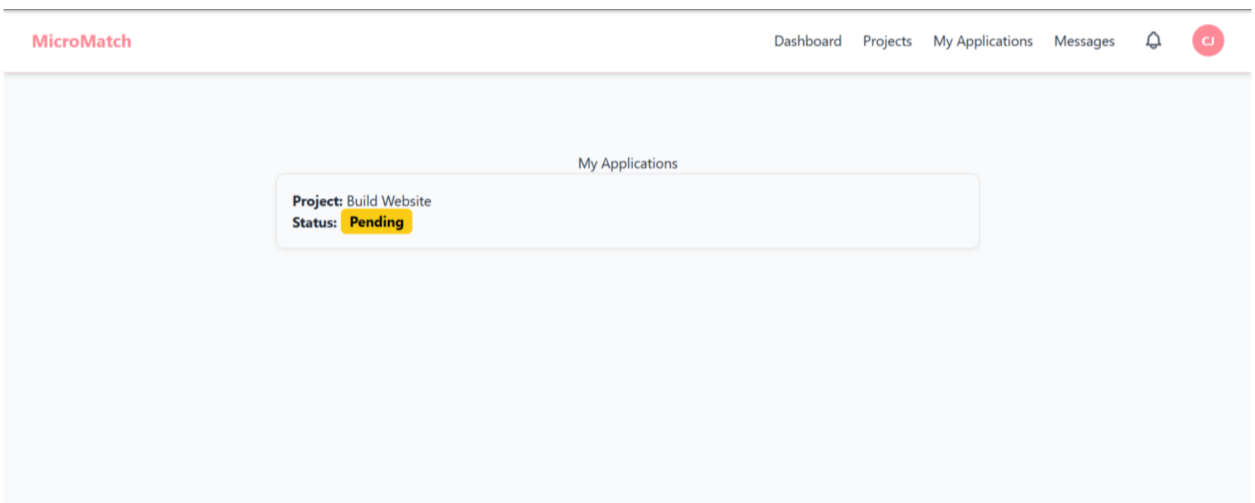


Smart recommendations on the student home dashboard. Match scores and factor breakdowns help students prioritize applications.

2.5 Application Workflow

2.5.1 Apply to a Project (Student)

1. Navigate to **Projects** and find a project you want to apply to.
2. Click the project card to open the detail view.
3. Click the **Apply** button.
4. Your application is submitted with status **Pending**.
5. The organization receives a notification of your application.
6. Track your application status under **My Applications** in the navigation.

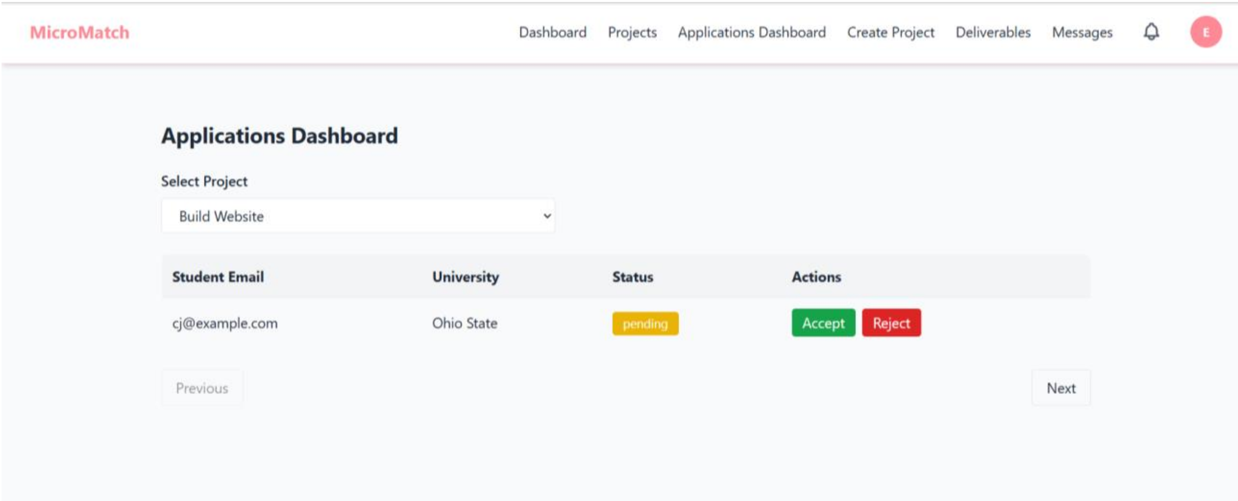


My Applications page. Status badges are color-coded: yellow = pending, green = accepted, red = rejected.

2.5.2 Review and Respond to Applications (Organization)

1. Click **Applications Dashboard** in the navigation bar.
2. Select the project from the **Select Project** dropdown.
3. All student applications appear in a table with email, university, and current status.

4. Click **Accept** to accept a student — they receive a notification immediately.
5. Click **Reject** to decline — the student is also notified.
6. Use the **Previous / Next** buttons to paginate through applications.

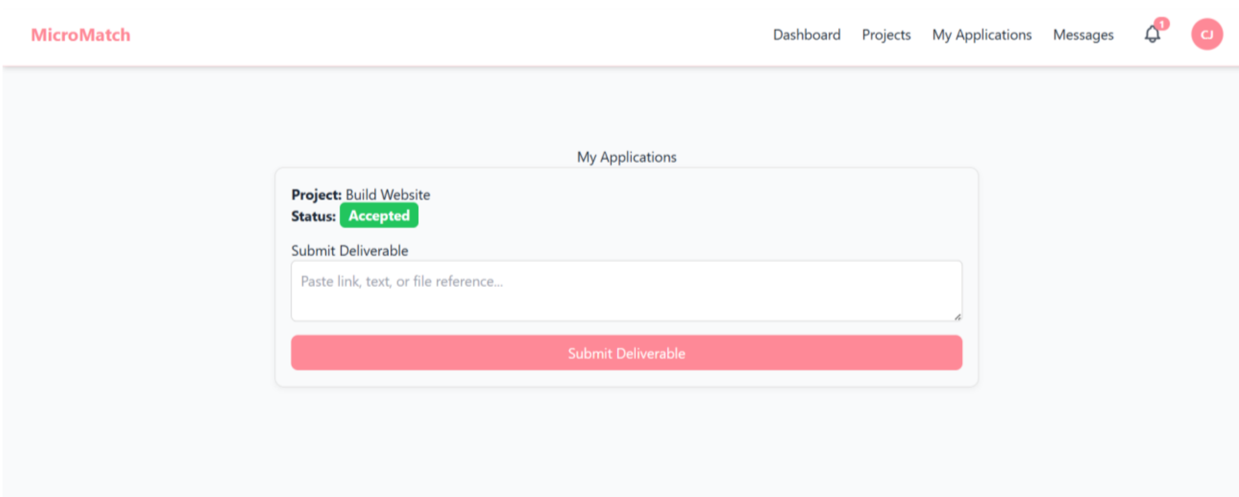


Organization Applications Dashboard. Only pending applications show Accept and Reject buttons.

2.5.3 Submit a Deliverable (Student)

Once your application is accepted, you can submit a deliverable representing your completed work.

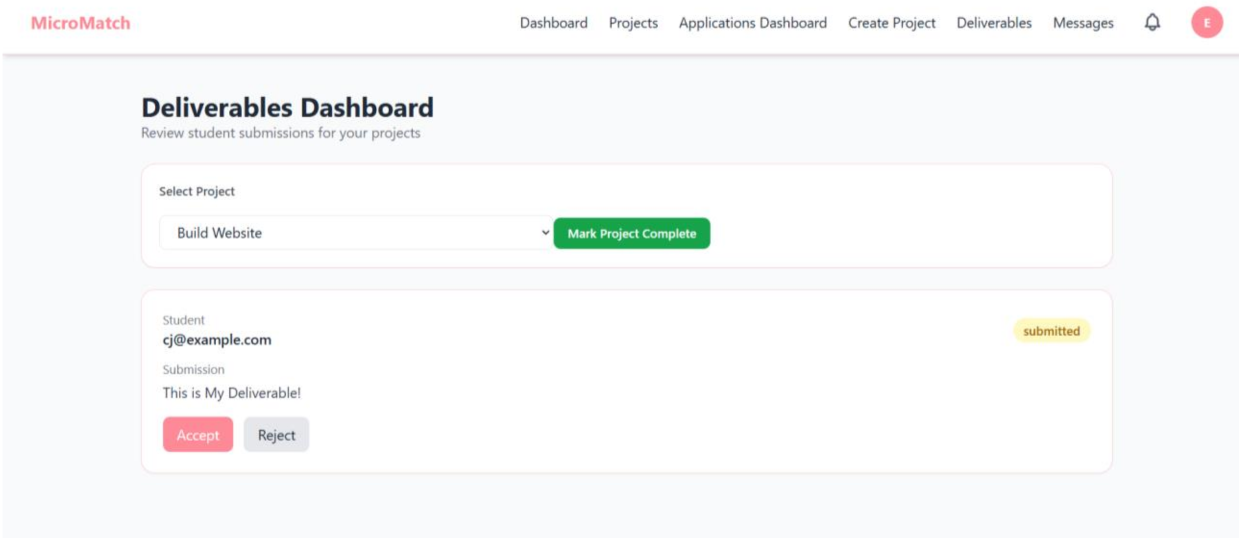
1. Navigate to **My Applications**.
2. Find the accepted project (green status badge).
3. Scroll to the **Submit Deliverable** section below the application card.
4. Paste a link, text summary, or file reference into the textarea.
5. Click **Submit Deliverable**.
6. The organization receives a notification that a deliverable is ready for review.



2.5.4 Review a Deliverable (Organization)

1. Click **Deliverables** in the navigation bar.
2. Review the submitted content.

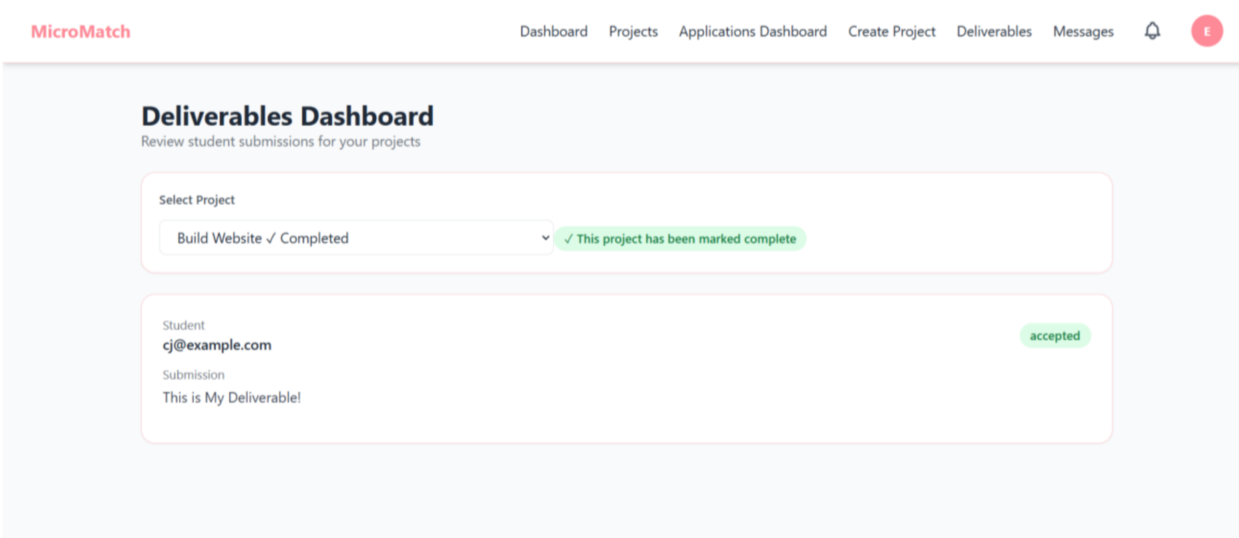
3. Enter optional **feedback text** in the feedback field.
4. Click **Approve** to accept, or **Request Revision** to ask for changes.



2.5.5 Mark a Project Complete (Organization)

Marking a project complete closes the project, triggers the feedback workflow, and automatically awards applicable achievement badges to the student.

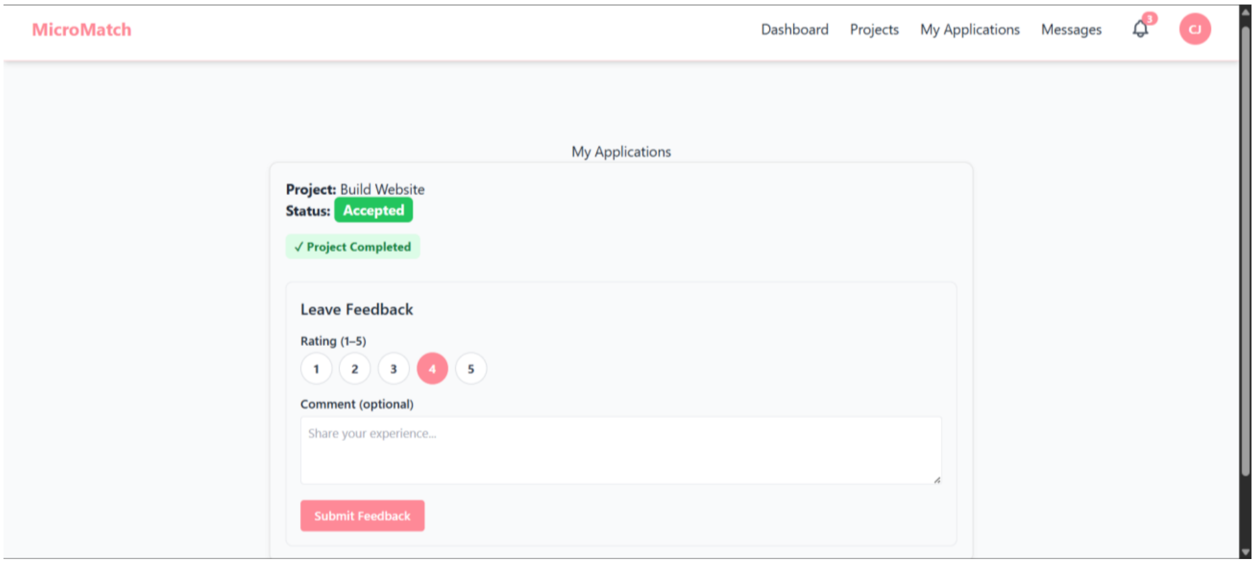
1. Navigate to your project listing or the Applications Dashboard.
2. Open the project detail view.
3. Click **Mark Complete**.
4. Confirm the action in the confirmation dialog.
5. The student receives a notification. The project status changes to **Completed**.
6. Both the student and the organization can now submit feedback.



2.6 Feedback System

After a project is marked complete, both parties can submit a 1-5 star rating and a written comment. Ratings feed into the student's average rating on their public profile and contribute to badge awards.

1. Navigate to **My Applications** (student) or the project detail (organization).
2. Find the completed project — it shows a green **Project Completed** banner.
3. Scroll to the **Feedback** section.
4. Click a **star rating** (1 to 5 stars).
5. Write an optional **comment** in the text field.
6. Click **Submit Feedback**.



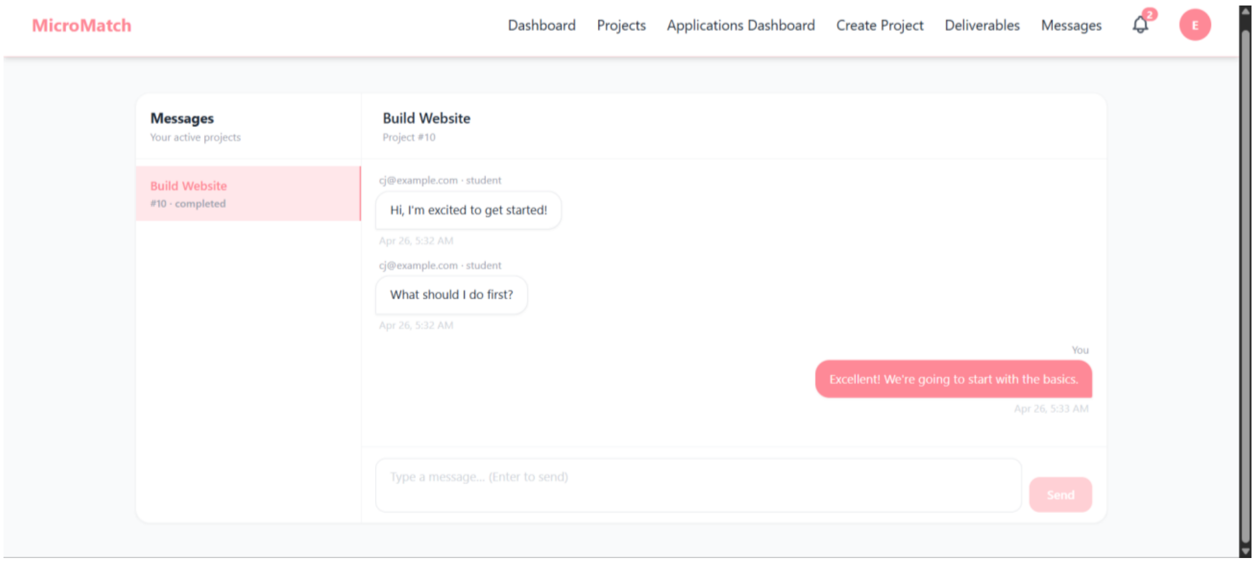
Feedback form shown on completed projects. Ratings are displayed as stars on the student's public profile and affect badge eligibility.

2.7 Project Messaging

MicroMatch provides a project-scoped messaging system. Only the accepted student and the owning organization can exchange messages within a project thread. All other users are blocked at the API level.

2.7.1 Access the Messages Hub

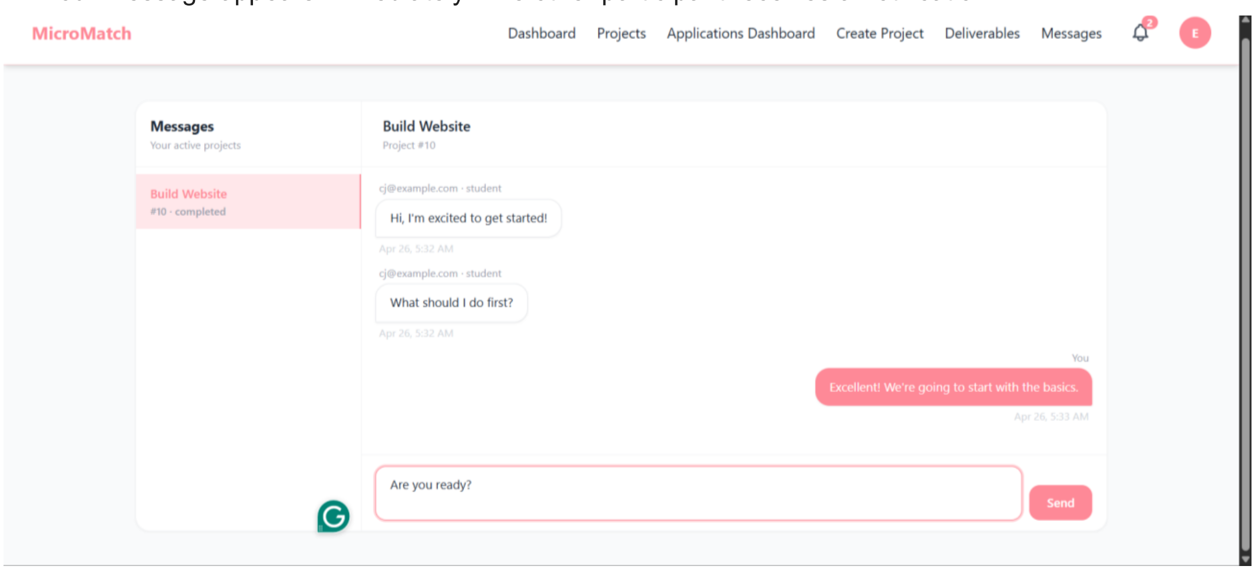
1. Click **Messages** in the navigation bar (visible to both students and organizations).
2. The **Messages Hub** opens with a **left sidebar** listing all projects you participate in.
3. Click any project name in the sidebar to open that conversation in the right panel.
4. Messages display oldest to newest. Your messages appear on the right; the other party's on the left.
5. The conversation auto-refreshes every **5 seconds** — no manual refresh needed.



Messages Hub. The left sidebar lists all accessible projects; the right panel shows the conversation.

2.7.2 Send a Message

1. Select a project from the left sidebar.
2. Type your message in the **text box** at the bottom of the chat panel.
3. Press **Enter** (without Shift) or click the **Send** button.
4. Your message appears immediately. The other participant receives a notification.



2.7.3 Messaging Access Rules

The messaging system enforces strict access control:

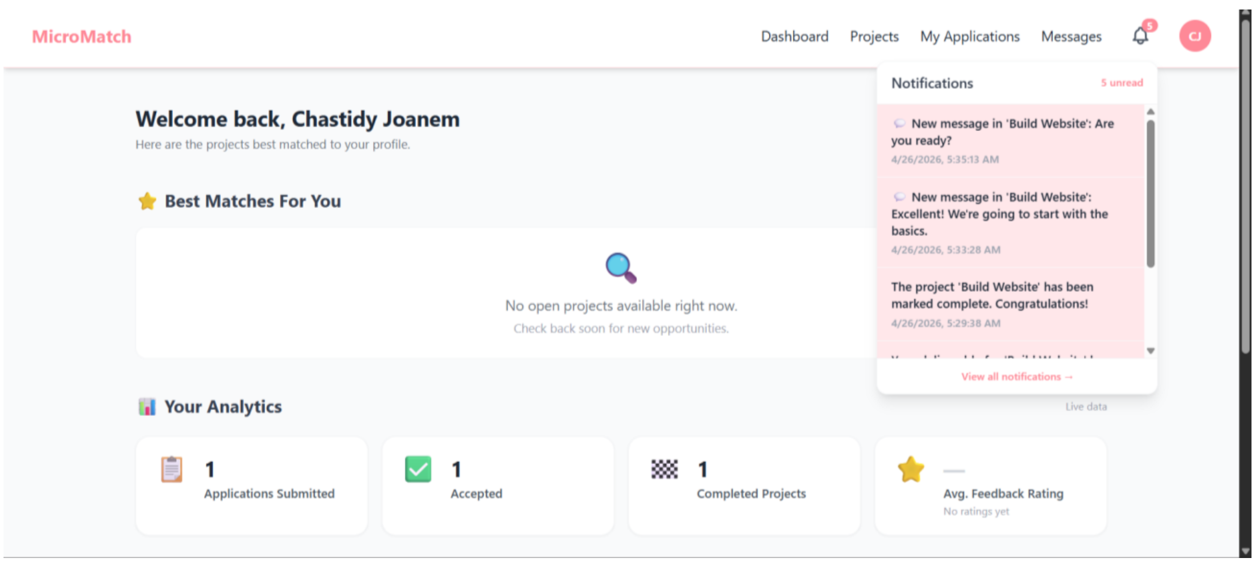
- Students with only a **pending application** cannot message — they must be accepted first.
- Students with a **rejected application** cannot access the project conversation.
- An organization that does **not own** the project cannot access its messages.
- Any unauthenticated or unrelated user sees a **locked screen** with a back button.

2.8 Notifications

The notification bell in the top navigation bar shows a badge count of unread notifications. Notifications are generated automatically for the following events:

Triggering Event	Who Is Notified
Student applies to a project	Organization (new application received)
Organization accepts an application	Student (you have been accepted!)
Organization rejects an application	Student (application was not selected)
Student submits a deliverable	Organization (deliverable ready for review)
Organization approves a deliverable	Student (deliverable approved)
Project is marked complete	Student (project is complete)
A new message is sent	The other project participant

1. Click the **bell icon** in the navigation bar to open the notification panel.
2. Unread notifications are highlighted. The badge count shows the number of unread items.
3. Click any notification to mark it as read — the badge count decrements.
4. Read notifications remain in the list but are no longer highlighted.



Notification bell showing unread count. Each notification links to the relevant project or action.

2.9 Analytics Dashboard

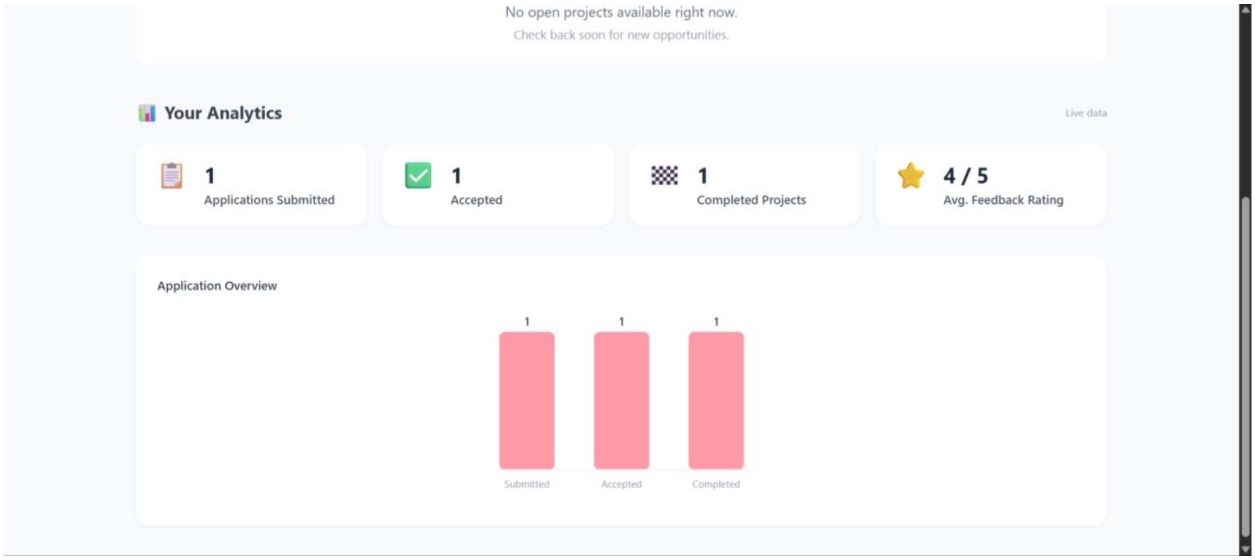
The analytics dashboard is embedded on the home page. It shows live productivity metrics computed directly from the database. Students see student metrics; organizations see organization metrics.

2.9.1 Student Analytics

Students see four stat cards and a bar chart:

- **Applications Submitted** — total applications ever made.
- **Accepted** — applications with accepted status.
- **Completed Projects** — accepted applications where the project is also marked complete.

- **Avg. Feedback Rating** — average star rating received. Shows 'No ratings yet' if none exist.

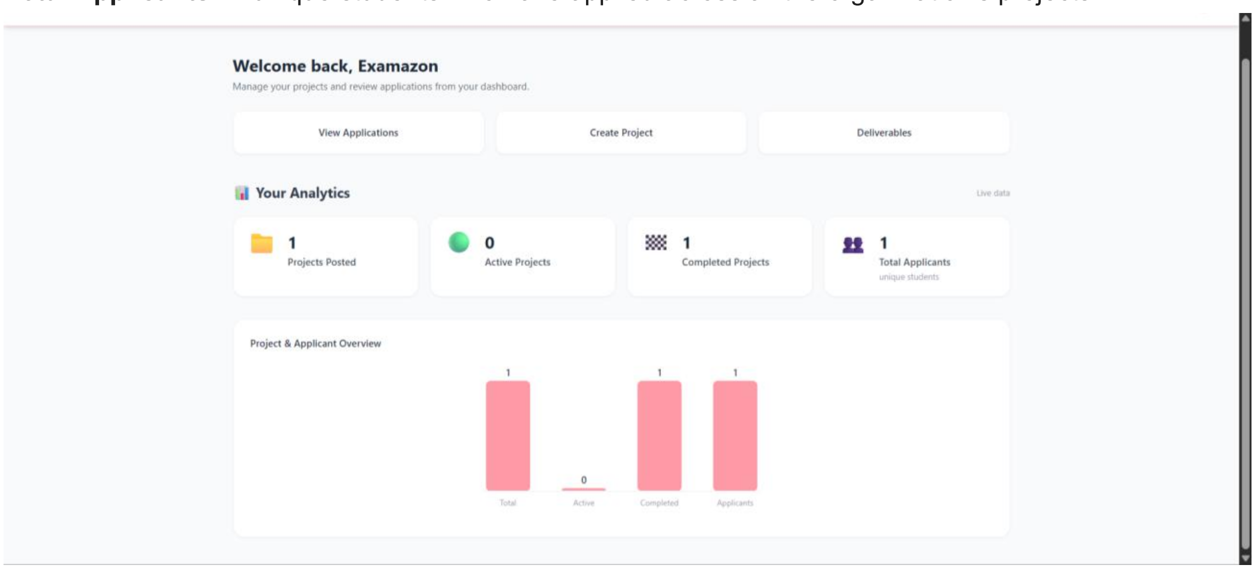


Student analytics dashboard. The bar chart visualizes submitted, accepted, and completed counts proportionally.

2.9.2 Organization Analytics

Organizations see four stat cards and a bar chart:

- **Projects Posted** — total projects ever created by this organization.
- **Active Projects** — projects currently with status 'open'.
- **Completed Projects** — projects with status 'completed'.
- **Total Applicants** — unique students who have applied across all the organization's projects.



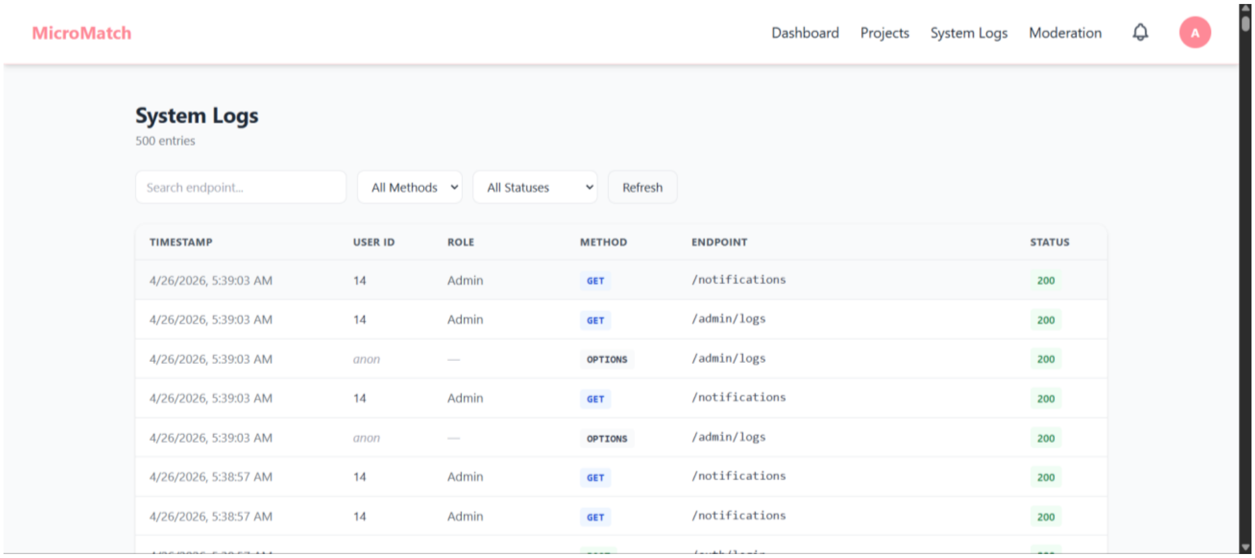
2.10 Administration (Admin Role Only)

2.10.1 System Logs

Every HTTP request to the API is logged automatically with full context for observability and debugging.

1. Log in as an Admin user.

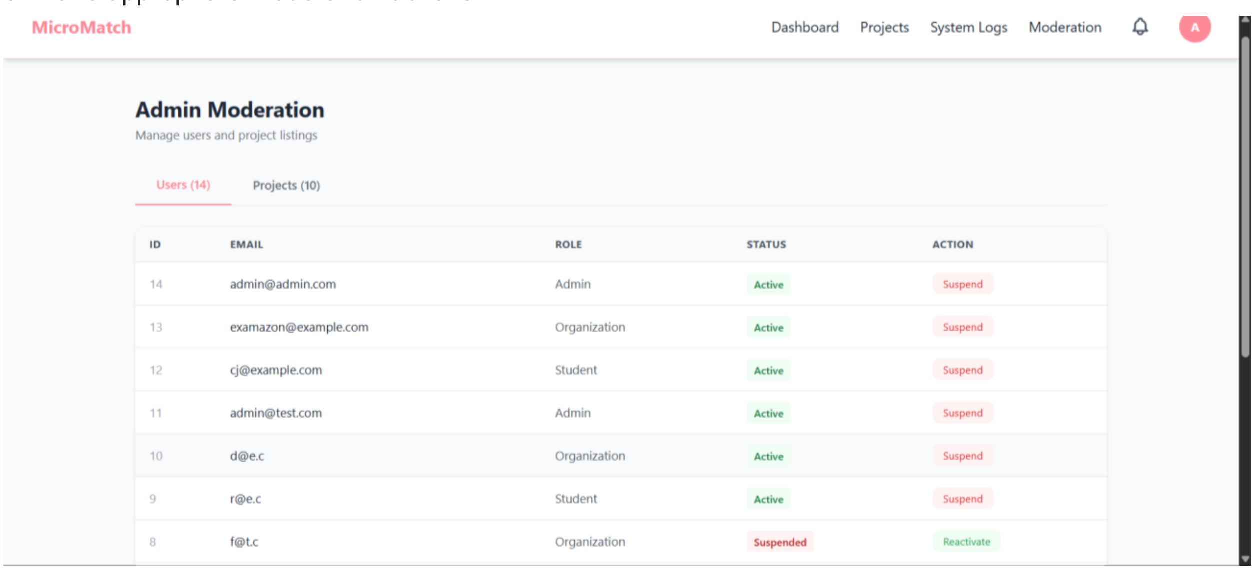
2. Click **System Logs** in the navigation bar.
3. The log table shows: timestamp, user id, role, endpoint, HTTP method, and response status code.
4. Use the available filters to narrow down logs by status code or endpoint path.



System Logs page. All API requests are recorded. 500-level errors are highlighted for quick identification.

2.10.2 Content Moderation

1. Click **Moderation** in the navigation bar (Admin only).
2. Review flagged content or user-reported items.
3. Take appropriate moderation actions.

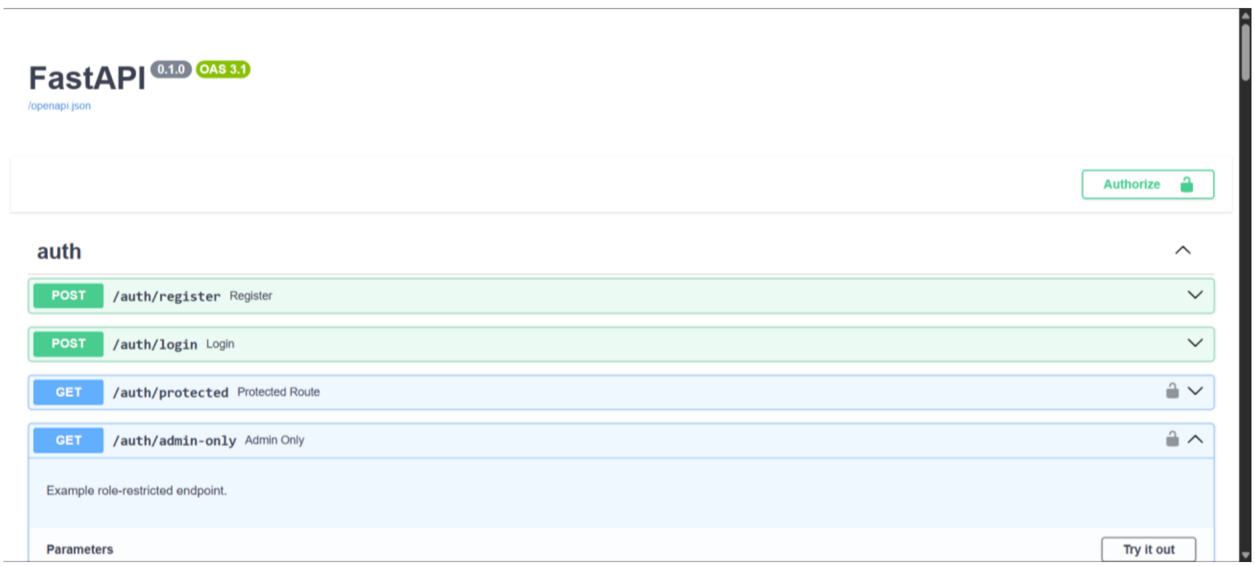


2.11 Troubleshooting Guide

Issue	Likely Cause	Resolution
localhost:3000 not loading	Frontend not started	Run: cd frontend && npm start
500 error on any API call	Backend not running or .env missing	Run: uvicorn app.main:app --reload in backend/
Database connection error on startup	PostgreSQL container not running	Run: docker compose up -d from project root
'column portfolio_links does not exist'	Schema migration not applied	Run the ALTER TABLE command from Step 5
Login returns 422 Unprocessable Entity	Sending form data instead of JSON	Use the application UI. The API expects JSON.
Messages page shows 403 Forbidden	Not an accepted participant in this project	Only accepted students and the owning org can message.
Badges not showing after completing a project	Student has no profile or trigger not fired	Ensure the student has a profile. Org must mark the project complete.
npm install fails	Node.js version too old	Upgrade to Node.js 18+ from nodejs.org

2.12 API Reference

The FastAPI backend provides automatically generated interactive documentation at http://localhost:8000/docs. All endpoints can be tested directly from this interface using the Authorize button.



FastAPI Swagger UI at localhost:8000/docs. All endpoints are organized by tag and documented with request/response schemas.

Tag	Key Endpoints	Auth Required
auth	POST /auth/register, POST /auth/login	No
projects	GET /projects, POST /projects, PATCH /projects/{id}/status	Yes

Tag	Key Endpoints	Auth Required
applications	POST /applications, GET /applications/me, PATCH /applications/{id}/status	Yes
deliverables	POST /applications/{id}/deliverables, GET /applications/{id}/deliverables	Yes
feedback	POST /feedback	Yes
messages	POST /projects/{id}/messages, GET /projects/{id}/messages	Yes (participant only)
notifications	GET /notifications, PUT /notifications/{id}/read	Yes
analytics	GET /analytics/student, GET /analytics/organization	Yes (RBAC enforced)
recommendations	GET /recommendations?top_n=10	Yes (student only)
student profile	POST/GET/PUT /student/profile, PUT /student/profile/enhance, GET /student/profile/{id}	Yes
organization profile	POST/GET/PUT /organization/profile	Yes (org only)